Deployment Guide — SGSITS GS-Website

Target: Debian 12 (Bookworm) bare-metal or VPS
Stack: Node.js 22 · MySQL 8 · PM2 · Nginx
Architecture: React SPA (static) + Express API (PM2) behind a single Nginx reverse proxy

Table of Contents

1. Pre-flight Checklist

Before starting, have the following ready:

- [ ] Root or `sudo` access to the Debian 12 server
- [ ] A domain/subdomain with an A record pointing to the server's IP
  (or use the bare IP if you are not using HTTPS yet)

- [ ] MySQL root password decided in advance
- [ ] A strong JWT secret — generate one now:
  ```
  bash     openssl rand -hex 32
  ```
- [ ] SMTP credentials if password-reset email is needed
  (Gmail → use an App Password, not your account password)
- [ ] Groq API key from console.groq.com if the AI chatbot is needed

2. System Dependencies

```
# Refresh package index and upgrade
sudo apt update && sudo apt upgrade -y

# Build tools (needed by some npm packages)
sudo apt install -y curl git unzip build-essential

# Node.js 22 LTS (via NodeSource official script)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Verify
node -v    # expected: v22.x.x
npm -v     # expected: 10.x.x

# PM2 — process manager
sudo npm install -g pm2

# Nginx
sudo apt install -y nginx

# MySQL 8.0
sudo apt install -y mysql-server
sudo systemctl enable mysql
sudo systemctl start mysql
```

3. MySQL Setup

3a. Secure the installation

```
sudo mysql_secure_installation
```

Answer the prompts:

- Set root password: **Yes** — choose a strong password
- Remove anonymous users: **Yes**
- Disallow remote root login: **Yes**
- Remove test database: **Yes**
- Reload privilege tables: **Yes**

3b. Create the application database and user

```
sudo mysql -u root -p
```

Inside the MySQL shell, run all four statements:

```
CREATE DATABASE college_website
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

CREATE USER 'sgsits'@'localhost' IDENTIFIED BY 'YourStrongDBPassword';

GRANT ALL PRIVILEGES ON college_website.* TO 'sgsits'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

> **Important:** The database name must be `college_website` exactly. The migration scripts reference this name directly.

4. Application User

Running the application as a dedicated non-root user limits the blast radius of any vulnerability.

```
sudo useradd -m -s /bin/bash sgsitsapp
sudo passwd sgsitsapp
```

Switch to this user for all remaining steps unless `sudo` is explicitly noted:

```
su - sgsitsapp
```

5. Upload the Project

Option A — Git (recommended)

```
git clone https://your-repo-url.git /home/sgsitsapp/gs-website
```

Option B — SCP from Windows

Run this on your Windows machine (PowerShell):

```
scp -r "C:\path\to\GS-Website" sgsitsapp@YOUR_SERVER_IP:/home/sgsitsapp/gs-website
```

The rest of this guide assumes the project lives at:

```
/home/sgsitsapp/gs-website/
├── backend/
└── sgsits-frontend/
```

6. Backend Setup

6a. Install dependencies

```
cd /home/sgsitsapp/gs-website/backend
npm install
```

6b. Create the production `.env`

```
nano /home/sgsitsapp/gs-website/backend/.env
```

Paste the block below and fill in every value marked `← change this`:

```
# ── Runtime ──────────────────────────────────────────────────────────
NODE_ENV=production
PORT=8000
APP_URL=https://yourdomain.ac.in           # ← change this

# ── Database ─────────────────────────────────────────────────────────
DB_HOST=localhost
DB_PORT=3306
DB_USER=sgsits
DB_PASSWORD=YourStrongDBPassword           # ← change this
DB_NAME=college_website

# ── Authentication ───────────────────────────────────────────────────
# Must be at least 32 random characters. Never reuse the dev value.
# Generate: openssl rand -hex 32
JWT_SECRET=replace_with_your_generated_secret   # ← change this
JWT_EXPIRES_IN=7d

# ── CORS / Frontend ──────────────────────────────────────────────────
# Must exactly match the browser Origin header — no trailing slash
CORS_ORIGIN=https://yourdomain.ac.in       # ← change this
FRONTEND_URL=https://yourdomain.ac.in      # ← change this

# ── Email (optional) ─────────────────────────────────────────────────
# Leave SMTP_USER empty to disable all outbound email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com             # ← change this (or leave blank)
SMTP_PASS=your-gmail-app-password          # ← change this (App Password, not login password)
SMTP_FROM=SGSITS Portal <no-reply@sgsits.ac.in>

# ── AI Chatbot (optional) ────────────────────────────────────────────
# Remove both lines if the RAG chatbot is not needed
GROQ_API_KEY=your_groq_api_key_here        # ← change this or remove
GROQ_MODEL=llama-3.3-70b-versatile
```

> **Security note:** The development `.env` that ships with the codebase contains a real SMTP password, a real Groq API key, and a weak JWT secret. Replace all three before going live.

6c. Create the uploads directory

The backend stores all uploaded files here. It must exist before the server starts.

```
mkdir -p /home/sgsitsapp/gs-website/backend/uploads
chmod 755 /home/sgsitsapp/gs-website/backend/uploads
```

Subdirectories (`uploads/faculty/` , `uploads/gallery/` , etc.) are created automatically on first upload.

6d. Run database migrations

```
cd /home/sgsitsapp/gs-website/backend

# Verify the DB connection before running anything
npm run db:test

# Apply all migrations — creates all 60+ tables
npm run db:migrate

# Load initial seed data (roles, departments, admin accounts)
npm run db:seed
```

> Seed data default password: **`Admin@123`**
> Change every seeded password immediately after first login.

Available database commands for reference:

Command	What it does
`npm run db:test`	Test DB connection
`npm run db:migrate`	Apply all pending migrations
`npm run db:seed`	Load enterprise seed data
`npm run db:fresh`	**Destructive** — wipe and re-seed (dev only)

6e. Smoke test the backend

```
node src/server.js
# Expected output: Server running on port 8000 [production]
```

Press `Ctrl+C` after confirming it starts. PM2 will manage the process going forward.

7. PM2 Process Manager

7a. Create the ecosystem config

```
nano /home/sgsitsapp/gs-website/backend/ecosystem.config.js
```

```
module.exports = {
  apps: [
    {
      name: 'sgsits-backend',
      script: './src/server.js',
      cwd: '/home/sgsitsapp/gs-website/backend',
      instances: 2,             // increase to 'max' to use all CPU cores
      exec_mode: 'cluster',
      watch: false,
      max_memory_restart: '512M',
      env_production: {
        NODE_ENV: 'production',
      },
      error_file: '/home/sgsitsapp/logs/backend-error.log',
      out_file:   '/home/sgsitsapp/logs/backend-out.log',
      log_date_format: 'YYYY-MM-DD HH:mm:ss',
    },
  ],
}
```

7b. Start and persist

```
# Create log directory
mkdir -p /home/sgsitsapp/logs

# Start the backend
cd /home/sgsitsapp/gs-website/backend
pm2 start ecosystem.config.js --env production

# Persist the process list across server reboots
pm2 save

# Register PM2 as a systemd service (run as root)
sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u sgsitsapp --hp /home/sgsitsapp
# PM2 prints an additional command — copy and run it exactly as printed
```

7c. Verify

```
pm2 status
# Expected: sgsits-backend  online  2 instances

pm2 logs sgsits-backend --lines 30
```

7d. Common PM2 commands

```
pm2 restart sgsits-backend     # restart after code change
pm2 reload sgsits-backend      # zero-downtime reload (cluster mode)
pm2 stop sgsits-backend        # stop
pm2 delete sgsits-backend      # remove from PM2
pm2 monit                      # real-time CPU/RAM monitor
pm2 logs sgsits-backend        # tail logs
```

8. Frontend Build

8a. Create the production env file

```
nano /home/sgsitsapp/gs-website/sgsits-frontend/.env.production
```

```
VITE_API_BASE_URL=https://yourdomain.ac.in/api
```

> The frontend axios client appends `/v1/<route>` to this value.
> A login call becomes `https://yourdomain.ac.in/api/v1/auth/login` .

8b. Install and build

```
cd /home/sgsitsapp/gs-website/sgsits-frontend
npm install
npm run build
```

This produces the static bundle at:

```
/home/sgsitsapp/gs-website/sgsits-frontend/dist/
```

Nginx serves this directory directly. No Node.js process is needed for the frontend.

9. Nginx Configuration

9a. Remove the default site

```
sudo rm -f /etc/nginx/sites-enabled/default
```

9b. Create the site config

```
sudo nano /etc/nginx/sites-available/sgsits
```

```
server {
    listen 80;
    server_name yourdomain.ac.in www.yourdomain.ac.in;

    # ── Security headers ──────────────────────────────────────
    add_header X-Frame-Options        "SAMEORIGIN"                    always;
    add_header X-Content-Type-Options "nosniff"                       always;
    add_header Referrer-Policy        "strict-origin-when-cross-origin"  always;

    # ── Gzip compression ──────────────────────────────────────
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types
        text/plain text/css application/json application/javascript
        text/xml application/xml image/svg+xml font/woff2;

    # ── Static frontend (React SPA) ───────────────────────────
    root /home/sgsitsapp/gs-website/sgsits-frontend/dist;
    index index.html;

    # Vite fingerprints JS/CSS filenames — safe to cache for 1 year
    location ~* \.(js|css|png|jpg|jpeg|gif|webp|svg|ico|woff|woff2|ttf)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # ── API — proxy to Express backend on port 8000 ───────────
    location /api/ {
        proxy_pass          http://127.0.0.1:8000/api/;
        proxy_http_version  1.1;
        proxy_set_header    Host              $host;
        proxy_set_header    X-Real-IP         $remote_addr;
        proxy_set_header    X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_read_timeout  120s;
        client_max_body_size 30M;   # must be >= backend multer limit (25 MB)
    }

    # ── Uploaded files — proxy to backend static handler ──────
    location /uploads/ {
        proxy_pass         http://127.0.0.1:8000/uploads/;
```

```
        proxy_http_version 1.1;
        proxy_set_header   Host $host;
        proxy_read_timeout 30s;
    }

    # ── React Router — all other paths fall back to index.html ──────────
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

> **Why `127.0.0.1:8000` and not `backend:8000` ?**
> The `nginx.conf` bundled in the repo uses a Docker container hostname (`backend`). On a bare-metal Debian server there is no Docker network, so the backend is reached via localhost.

9c. Enable the site and reload

```
sudo ln -s /etc/nginx/sites-available/sgsits /etc/nginx/sites-enabled/sgsits

# Validate config — must print "syntax is ok" and "test is successful"
sudo nginx -t

sudo systemctl reload nginx
```

10. HTTPS with Let's Encrypt

Skip this section only if you are deploying to an internal network without a public domain.

```
sudo apt install -y certbot python3-certbot-nginx

sudo certbot --nginx -d yourdomain.ac.in -d www.yourdomain.ac.in
```

Certbot will:

1. Obtain a certificate
2. Automatically edit the nginx config to add port 443 and an HTTP→HTTPS redirect
3. Install a systemd timer for automatic renewal

Verify the renewal timer is active:

```
sudo systemctl status certbot.timer
```

After certbot completes, update environment variables to use `https://` :

```
# Backend .env
APP_URL=https://yourdomain.ac.in
CORS_ORIGIN=https://yourdomain.ac.in
FRONTEND_URL=https://yourdomain.ac.in

# Frontend .env.production
VITE_API_BASE_URL=https://yourdomain.ac.in/api
```

Then rebuild the frontend and restart the backend:

```
cd /home/sgsitsapp/gs-website/sgsits-frontend && npm run build
pm2 restart sgsits-backend
sudo nginx -s reload
```

11. Firewall

```
sudo apt install -y ufw

sudo ufw allow OpenSSH          # keep SSH access
sudo ufw allow 'Nginx Full'     # opens port 80 (HTTP) and 443 (HTTPS)
sudo ufw enable

sudo ufw status
```

> Do **not** open port 8000. The backend must only be reachable through Nginx, never directly from the internet.

12. File Permissions

Nginx runs as `www-data`. It needs read access to the frontend dist and execute access to traverse the home directory.

```
# Allow www-data to traverse into the app user's home
chmod o+x /home/sgsitsapp

# Allow www-data to read all dist files
chmod -R o+r /home/sgsitsapp/gs-website/sgsits-frontend/dist

# Uploads must be writable by the Node.js process (sgsitsapp user)
chown -R sgsitsapp:sgsitsapp /home/sgsitsapp/gs-website/backend/uploads
chmod -R 755 /home/sgsitsapp/gs-website/backend/uploads
```

13. Environment Variable Reference

Backend — `/home/sgsitsapp/gs-website/backend/.env`

Variable	Required	Description
`NODE_ENV`	✅	Must be `production`
`PORT`	✅	`8000` — the port Express listens on
`APP_URL`	✅	Full public URL of the site, e.g. `https://yourdomain.ac.in`
`DB_HOST`	✅	`localhost`
`DB_PORT`	✅	`3306`
`DB_USER`	✅	`sgsits` (the user created in step 3)
`DB_PASSWORD`	✅	Your chosen DB password
`DB_NAME`	✅	`college_website` — must match exactly
`JWT_SECRET`	✅	Min 32 random chars — `openssl rand -hex 32`
`JWT_EXPIRES_IN`	✅	Token lifetime, e.g. `7d`, `24h`

Variable	Required	Description
`CORS_ORIGIN`	✅	Frontend origin, e.g. `https://yourdomain.ac.in` — no trailing slash
`FRONTEND_URL`	✅	Same as `CORS_ORIGIN` — used in email links
`SMTP_HOST`	Optional	SMTP server hostname
`SMTP_PORT`	Optional	`587` (STARTTLS) or `465` (SSL)
`SMTP_USER`	Optional	SMTP account — leave blank to disable email
`SMTP_PASS`	Optional	Gmail App Password (Settings → Security → App Passwords)
`SMTP_FROM`	Optional	Display name + address for outbound mail
`GROQ_API_KEY`	Optional	From console.groq.com — required for AI chatbot only
`GROQ_MODEL`	Optional	`llama-3.3-70b-versatile` or any Groq-supported model

Frontend — `/home/sgsitsapp/gs-website/sgsits-frontend/.env.production`

Variable	Required	Description
`VITE_API_BASE_URL`	✅	`https://yourdomain.ac.in/api` — must end in `/api`, no trailing slash

14. Post-Deployment Checklist

Run through these after every fresh deployment:

```
# 1. API health check
curl https://yourdomain.ac.in/api/v1/health
# Expected: {"status":"ok","timestamp":"..."}

# 2. Backend process status
pm2 status
# Expected: sgsits-backend  online

# 3. Backend logs (look for errors)
pm2 logs sgsits-backend --lines 50

# 4. Nginx status
sudo systemctl status nginx

# 5. MySQL status
sudo systemctl status mysql

# 6. Certificate expiry (if HTTPS enabled)
sudo certbot certificates
```

Manual browser checks:

- [ ] Home page loads — `https://yourdomain.ac.in`
- [ ] Login page loads — `https://yourdomain.ac.in/login`
- [ ] Staff login succeeds with seeded credentials
- [ ] After login, navbar shows **Dashboard** instead of **Login**
- [ ] Dashboard redirects to the correct role-specific portal
- [ ] An uploaded file (e.g. profile photo) displays correctly
- [ ] **Change all seeded passwords** (default: `Admin@123`)

15. Redeploy / Update Procedure

Code-only update (no migration, no dependency change)

```
cd /home/sgsitsapp/gs-website
git pull

# Restart backend (zero-downtime in cluster mode)
pm2 reload sgsits-backend

# Rebuild and serve new frontend
cd sgsits-frontend
npm run build
sudo nginx -s reload
```

Update with new dependencies

```
cd /home/sgsitsapp/gs-website/backend
git pull
npm install
pm2 reload sgsits-backend

cd ../sgsits-frontend
npm install
npm run build
sudo nginx -s reload
```

Update with database migrations

```
cd /home/sgsitsapp/gs-website/backend
git pull
npm install
npm run db:migrate     # always safe to re-run; migrations are idempotent
pm2 reload sgsits-backend

cd ../sgsits-frontend
npm install
npm run build
sudo nginx -s reload
```

16. Troubleshooting

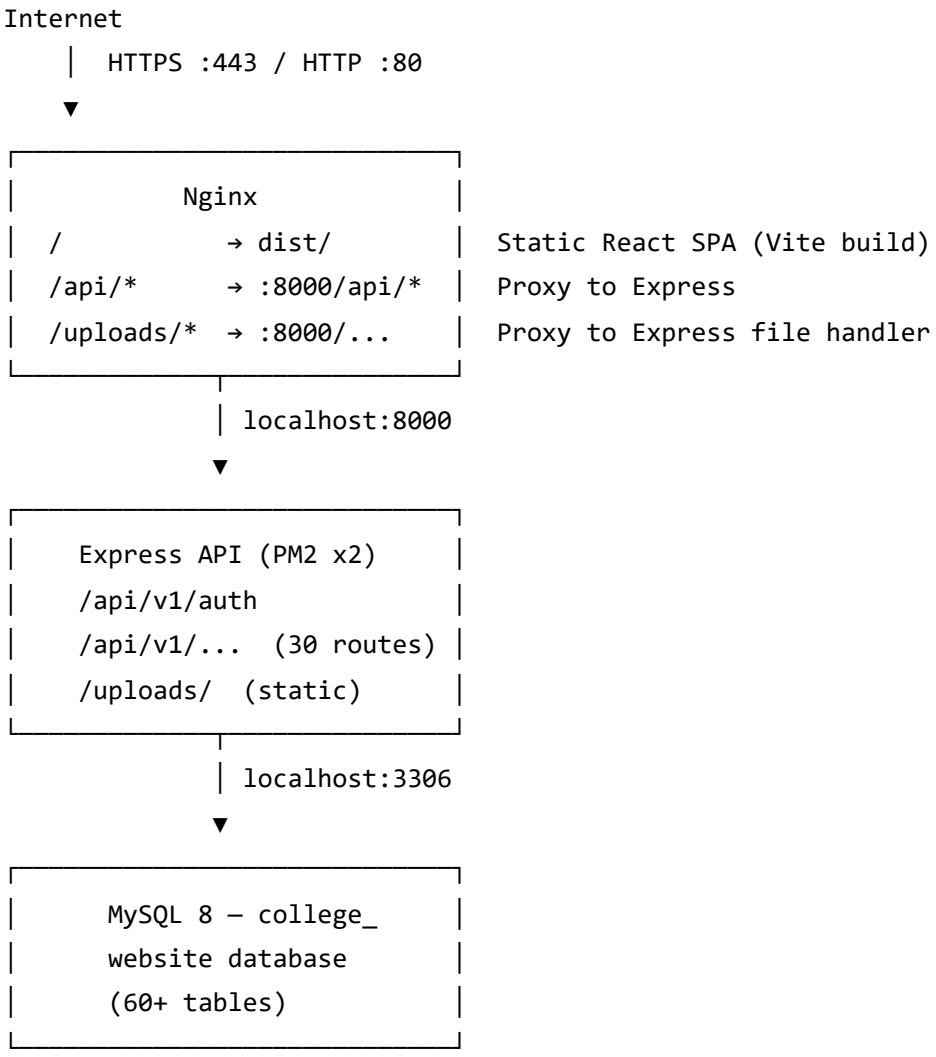
Symptom	Most likely cause	Fix
502 Bad Gateway	Backend is down	`pm2 status` → `pm2 restart sgsits-backend`; check `pm2 logs`
`/uploads/` images return 404	Backend not running or wrong proxy target	Verify `pm2 status`; confirm nginx `proxy_pass` points to `127.0.0.1:8000`
CORS error in browser console	`CORS_ORIGIN` mismatch	Value must exactly match the browser Origin (include `https://`, no trailing slash)
Login returns 401 immediately	JWT secret mismatch between restarts	Ensure `.env` file is stable; signing and verification must use the same secret
413 Request Entity Too Large	nginx body size limit too small	Confirm `client_max_body_size 30M;` is inside the `/api/` location block
Rate-limit responses show wrong IP	`trust proxy` misconfigured	Already set in `app.js` — ensure nginx forwards `X-Forwarded-For` header (the config above does this)
PM2 processes gone after reboot	Startup hook not registered	Re-run `pm2 startup`, run the printed command, then `pm2 save`
Database migration fails	DB user missing privileges or wrong DB name	Verify the `GRANT ALL` statement in step 3; confirm `DB_NAME=college_website` in `.env`
Blank page after deploy	Stale frontend build	`npm run build` in `sgsits-frontend/`, then `sudo nginx -s reload`

Symptom	Most likely cause	Fix
Email not sending	SMTP credentials wrong or App Password not set up	Test with `node -e "require('./src/services/email.service')"` or disable email by clearing `SMTP_USER`

Useful log locations

Log	Command
Backend stdout	`pm2 logs sgsits-backend`
Backend stderr	`pm2 logs sgsits-backend --err`
Nginx access log	`sudo tail -f /var/log/nginx/access.log`
Nginx error log	`sudo tail -f /var/log/nginx/error.log`
MySQL error log	`sudo tail -f /var/log/mysql/error.log`
System journal	`sudo journalctl -u nginx -u mysql -f`

Architecture Overview



Port 8000 is **not** open to the internet — all traffic flows through Nginx on 80/443.

Last updated: June 2026 — Debian 12 · Node.js 22 · MySQL 8.0 · PM2 5.x · Nginx 1.24